

ActionPiece: 面向生成式推荐的 上下文感知动作序列词元化

Yupeng Hou^{1*} Jianmo Ni² Zhankui He² Naveen Sachdeva² Wang-Cheng Kang²
Ed H. Chi² Julian McAuley¹ Derek Zhiyuan Cheng²

摘要

生成式推荐 (Generative Recommendation, GR) 是一种新兴范式: 它将用户动作词元化为离散的词元模式, 并以自回归方式生成预测结果。然而, 现有 GR 模型独立地词元化每个动作, 在所有序列中为相同动作分配相同的固定词元, 而不考虑上下文关系。这种上下文感知能力的缺失可能导致次优性能, 因为同一动作会因其周围上下文不同而具有不同含义。为解决这一问题, 我们提出 ActionPiece, 在动作序列词元化时显式引入上下文。在 ActionPiece 中, 每个动作由物品特征的一个集合表示。给定动作序列语料库, 我们根据特征模式在单个集合内部以及相邻集合之间的共现频率, 将其合并为新词元, 从而构建词表。考虑到特征集合的无序性, 我们进一步引入集合置换正则化, 为语义相同的动作序列生成多种分词结果。代码见: https://github.com/google-deepmind/action_piece。

*本工作于作者在 Google DeepMind 担任学生研究员期间完成。¹加利福尼亚大学圣迭戈分校 ²Google DeepMind. Correspondence to: Yupeng Hou and Jianmo Ni <yphou@ucsd.edu, jianmon@google.com>.

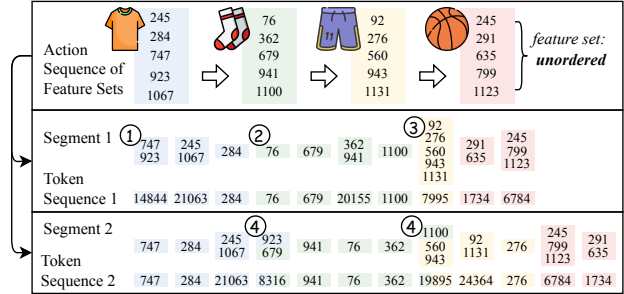


Figure 1. ActionPiece 词元化过程示意图。每个动作表示为无序特征集合。图中展示了两种可能的词元化序列, 其中各特征被划分为不同片段。同一动作可根据周围上下文被词元化为不同词元。详细案例分析见 Section 4.5。

1. 引言

生成式推荐 (Generative Recommendation, GR) (Tay et al., 2022; Rajput et al., 2023; Zheng et al., 2024; Zhai et al., 2024) 是序列推荐任务 (Hidasi et al., 2016; Kang & McAuley, 2018) 中的一种新兴范式。GR 模型将用户动作 (通常由交互物品表示) 词元化为离散词元, 并学习以自回归方式生成词元, 再将其解析为推荐物品。这些词元共享一个不随物品池规模增长的紧凑词表, 从而提升模型可扩展性、内存效率和推荐性能。按时间顺序组织用户历史交互的输入动作序列, 对于理解用户意图至关重要 (Hidasi et al., 2016; Li et al., 2017; Kang & McAuley, 2018)。同一动作 (e.g., 购买同一件物品) 在不同动作序列中可能具有不同含义。某一动作的依据可从上下文中找到, 例如序列中的其他物品是否具有相同品牌、色调或价格区间 (Zhang et al., 2019; Zhou et al., 2020; Hou et al., 2022; 2023; Yuan et al., 2023)。

尽管动作之间的上下文关系十分重要, 现有方法仍

脱离上下文独立词元化每个动作（汇总见 Table 1）。典型的动作序列词元化流程包括两步：（1）将每个动作/物品独立词元化为一个词元模式；（2）用相应词元模式替换输入序列中的每个动作。如此得到的词元并不显式包含上下文，而是完全依赖充分训练的自回归模型参数来泛化并理解上下文，这对 GR 模型的能力提出了挑战。作为对比，语言建模中的词元化同样起源于词级词元化等上下文无关方法 (Sutskever et al., 2014; Bahdanau et al., 2015)。经过十年发展，现代大语言模型 (Large Language Models, LLMs) (OpenAI, 2022; Anil et al., 2023; Touvron et al., 2023; Zhao et al., 2023) 的大多数词元化方法已采用上下文感知方案，包括 BPE (Sennrich et al., 2016) 和 Unigram 词元化 (Kudo, 2018)；它们会将相同词根连同相邻上下文词元化为不同词元。

本文旨在迈出动作序列建模中上下文感知词元化的第一步。类似于字符或字节作为语言建模的基本单元，我们将物品的关联特征视为初始词元。核心思想是在训练动作序列中迭代寻找最常共现的词元对，再将其合并为表示上下文片段的新词元。然而，实现这一思想并非易事：文本中的字符天然构成序列，而动作的关联特征却构成无序集合 (Zhang et al., 2019; Zhou et al., 2020)。因此，所提出的词元化算法应作用于词元集合序列。我们需要仔细考虑应统计哪些词元对、它们位于单个集合内部还是两个相邻集合之间，以及应为这些不同关系赋予多大权重。

为此，我们提出 ActionPiece，使相同动作能够根据周围上下文被词元化为不同词元。（1）**词表构建**首先初始化词表，将每个唯一特征作为初始词元。随后通过迭代学习合并规则构建词表，每条合并规则规定一对词元可合并为一个新词元。在每轮迭代中，我们遍历训练语料库，统计已有词元的共现次数。考虑词元对的结构差异，e.g., 它们出现在单个集合内部还是两个相邻集合之间，我们为不同词元对赋予不同权重。（2）**分词**是指将动作序列中的原始特征划分为若干组，并用词表中的词元替换。为充分利用每个动作特征集合的无序性，我们引入集合置换正则化。通过随机置换每个集合内的特征，可为同

Table 1. 生成式推荐中不同动作词元化方法的对比。“上下文感知”表示相同动作能否根据周围上下文被词元化为不同词元；“无序”表示是否以顺序无关方式使用物品特征或语义 ID。

动作词元化	示例	上下文感知	无序
乘积量化	VQ-Rec (Hou et al., 2023)	否	是
层次聚类	P5-CID (Hua et al., 2023)	否	否
残差量化	TIGER (Rajput et al., 2023)	否	否
文本词元化	LMIndexer (Jin et al., 2024)	否	否
原始特征	HSTU (Zhai et al., 2024)	否	否
SentencePiece	SPM-SID (Singh et al., 2024)	否	否
ActionPiece	本文	是	是

一动作序列生成多个语义不变的字元序列。这些变体既能作为训练数据的自然增强，也能在模型推理期间实现内生集成。

2. 相关工作

生成式推荐。传统序列推荐模型通常依赖大型嵌入表存储所有物品的表示，带来了显著的工程和优化挑战 (Hidasi et al., 2016; Li et al., 2017; Kang & McAuley, 2018)。生成式推荐 (Rajput et al., 2023; Zheng et al., 2024; Zhai et al., 2024; Deldjoo et al., 2024; Hou et al., 2025) 将每个物品词元化为共享词表中的词元，以解决上述问题。这种生成范式通过自回归生成后续词元作为推荐结果，具有内存高效 (Rajput et al., 2023; Yang et al., 2024; Ding et al., 2024)、可扩展 (Zhai et al., 2024; Liu et al., 2024b) 以及更易与 LLM 对齐 (Zheng et al., 2024; Jin et al., 2024; Tan et al., 2024; Li et al., 2025) 等优势。现有研究提出了多种动作词元化技术，例如层次聚类 (Hua et al., 2023; Si et al., 2024)、量化 (Rajput et al., 2023; Wang et al., 2024a; Zhu et al., 2024a)，或与推荐模型联合训练 (Liu et al., 2025)。其他工作还引入协同过滤 (Petrov & Macdonald, 2023; Wang et al., 2024c;b; Liu et al., 2024b;a)、自然语言 (Zheng et al., 2024; Jin et al., 2024; Hou et al., 2024b; Zhang et al., 2025a) 等额外模态。然而，当前方法均独立词元化每个动作，忽略周围上下文。本文提出首个上下文感知动作词元化方法，使相同动作在不同动作序列中被词元化为不同结果。

Algorithm 1 ActionPiece 词表构建

```

input 序列语料库  $S'$ , 初始词元  $\mathcal{V}_0$ , 目标大小  $Q$ 
output 合并规则  $\mathcal{R}$ , 构建的词表  $\mathcal{V}$ 
1: 初始化词表  $\mathcal{V} \leftarrow \mathcal{V}_0$  # 每个初始词元对应一个唯一物品特征
2:  $\mathcal{R} \leftarrow \emptyset$ 
3: while  $|\mathcal{V}| < Q$  do
4:   # 统计: 累积加权词元共现
5:    $\text{count}(\cdot, \cdot) \leftarrow \text{Count}(S', \mathcal{V})$  # Algorithm 2
6:   # 更新: 将高频词元对合并为新词元
7:   选择  $(c_u, c_v) \leftarrow \arg \max_{(c_i, c_j)} \text{count}(c_i, c_j)$ 
8:    $S' \leftarrow \text{Update}(S', \{(c_u, c_v) \rightarrow c_{\text{new}}\})$  # Algorithm 3
9:    $\mathcal{R} \leftarrow \mathcal{R} \cup \{(c_u, c_v) \rightarrow c_{\text{new}}\}$  # 新合并规则
10:   $\mathcal{V} \leftarrow \mathcal{V} \cup \{c_{\text{new}}\}$  # 向词表添加新词元
11: end while
返回  $\mathcal{R}, \mathcal{V}$ 

```

语言建模中的词元化。词元化是将原始文本转换为离散词元序列的过程 (Kudo & Richardson, 2018)。早期词级方法与上下文无关, 难以处理未登录词 (Sutskever et al., 2014; Bahdanau et al., 2015), 因此子词级词元化逐渐成为主流选择。这些子词词元化器的词表以迭代方式构建: 可以自底向上 (从小词表出发, 将频繁出现的词元对合并为新词元) (Wu et al., 2016; Sennrich et al., 2016), 也可以自顶向下 (从大词表出发, 裁剪词元以最小化似然下降) (Kudo, 2018; Yehezkel & Pinter, 2023)。词表构建完成后, 可使用构建词表时的同一方法或基于额外目标对文本进行分词 (He et al., 2020; Provilkov et al., 2020; Hofmann et al., 2022; Schmidt et al., 2024)。类比而言, 现有动作词元化器与上下文无关, 其作用类似于“词级”语言词元化器。本文迈出了构建上下文感知、子动作级动作词元化器的第一步。

3. 方法

本节介绍 ActionPiece, 一种面向生成式推荐的上下文感知动作序列词元化方法。首先在 Section 3.1 中形式化任务; 随后在 Section 3.2 中介绍词表构建与分词过程; 最后在 Section 3.3 中说明如何使用 ActionPiece 词元化序列进行模型训练与推理。

3.1. 问题定义

给定按时间戳排列的用户历史动作 $S = \{i_1, i_2, \dots, i_t\}$, 任务是预测用户接下来将交互的物品 i_{t+1} 。

将动作表示为无序特征集合。现代推荐系统通常为每个物品 i_j 关联一个特征集合 $\mathcal{A}_j$ (Zhang et al., 2019; Zhou et al., 2020; Cheng et al., 2016)。假设每个物品有 m 个特征, 物品 i_j 的第 k 个特征记为 $f_{j,k} \in \mathcal{F}_k$, 其中 $\mathcal{F}_k$ 是第 k 个特征所有可能取值的集合。与使用有序语义 ID (e.g., RQ-VAE (Rajput et al., 2023; Singh et al., 2024) 生成的 ID) 表示动作相比, 无序集合设定有两项关键优势: (1) 无需指定特征顺序, 更符合大多数推荐系统中物品或动作的表示方式; (2) 可纳入更通用的离散与数值特征, 例如类别、品牌和价格 (Pazzani & Billsus, 2007; Juan et al., 2016)。

将动作序列表示为集合序列。将每个物品表示为无序集合后, 输入动作序列可写为 $S' = \{\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_t\}$, 即按时间排序的集合序列。每个集合内部不存在顺序, 但来自不同集合的特征之间具有顺序。词元化器设计应同时考虑特征之间的有序与无序关系。

生成式推荐任务。本文旨在设计一个词元化器, 将输入动作序列 S' 映射为词元序列 $C = \{c_1, c_2, \dots, c_l\}$, 其中 l 表示序列中的词元数, 且通常大于动作数 t 。随后训练 GR 模型, 自回归生成词元 $\{c_{l+1}, \dots, c_q\}$, 并将其解析为下一物品预测 $\hat{i}_{t+1}$ 。

3.2. 上下文感知动作序列词元化器

所提出的词元化器将动作序列 (表示为特征集合序列) 转换为词元序列。在 ActionPiece 词元化序列中, 每个词元对应一个包含不同数量特征的集合。例如, 一个词元可表示: (1) 单个物品的部分特征; (2) 单个特征; (3) 单个物品的全部特征; 或 (4) 多个物品的特征。这四类词元亦标注于 Figure 1。下面首先说明如何由动作序列语料库构建 ActionPiece 词元化器的词表 (Section 3.2.1), 随后介绍如何将动作序列切分为新的集合序列, 其中每个集合对应构

建词表中的一个词元 (Section 3.2.2)。

3.2.1. 基于动作序列语料库的词表构建

给定动作序列语料库 $\mathcal{S}'$ ，词表构建的目标是创建包含 Q 个词元的词表 $\mathcal{V}$ 。每个词元表示语料库中频繁出现的一种特征组合。与 BPE (Sennrich et al., 2016) 类似，我们采用自底向上的方法构建词表。该过程从词元的**初始词表** $\mathcal{V}_0$ 开始，以迭代方式进行，每轮向词表添加一个新词元，直至达到预设目标大小。每轮包含两个连续步骤：**统计**，识别出现最频繁的词元对；**更新**，将选中的词元对合并为新词元并修改语料库。算法流程见 Algorithm 1。

词表初始化。在 BPE 中，每个词元表示一个字节序列，因而最基本的单元——初始词元——是单字节，它们构成 BPE 的初始词表。类似地，ActionPiece 中的每个词元表示一个特征集合。因此，我们用如下词表初始化 ActionPiece：每个词元表示仅包含一个唯一物品特征的集合。形式化地，初始词表记为 $\mathcal{V}_0 = \{c = \{f\} | f \in \mathcal{F}_1 \cup \dots \cup \mathcal{F}_m\}$ 。词表初始化后，每个动作序列（特征集合序列）均可表示为词元集合序列。

统计：上下文感知的词元共现统计。词表构建的每轮迭代首先统计语料库中词元对的共现。这些词元对捕获重要的特征组合，并通过创建新词元对其编码。集合序列中存在两类词元共现：(1) 两个词元位于同一集合；(2) 两个词元位于序列中的相邻集合。值得注意的是，第二类使 ActionPiece 能显式纳入上下文信息。

加权共现统计。在一维词元序列 (e.g., 文本) 中，所有词元对通常被同等对待。然而在词元集合序列中，词元对会因其类型及所在集合大小而异。为考虑这些差异，我们为不同词元对分配不同权重。具体而言，随机置换每个集合中的词元并将其展平为单一词元序列，以此建立词元集合序列与词元序列之间的联系。令 $P(c, c')$ 表示词元 c 与 c' 在展平序列中相邻的期望概率。对于来自同一集合的两个词元，

有：

$$P(c_1, c_2) = P(c_2, c_1) = \frac{|\mathcal{A}_i| - 1}{\binom{|\mathcal{A}_i|}{2}} = \frac{2}{|\mathcal{A}_i|}, \quad c_1, c_2 \in \mathcal{A}_i, \quad (1)$$

对于来自相邻集合的两个词元，有：

$$P(c_1, c_3) = \frac{1}{|\mathcal{A}_i| \times |\mathcal{A}_{i+1}|}, \quad c_1 \in \mathcal{A}_i, c_3 \in \mathcal{A}_{i+1}. \quad (2)$$

将展平序列中所有相邻词元对的概率视为 1，则原始词元集合序列中词元对的权重对应 Equations (1) and (2) 给出的概率。示意图 Figure 2。

累积共现权重。上述权重仅根据共现类型与集合大小计算，而不考虑具体词元。词元 c_i 与 c_j 可能在一个序列中位于同一集合，却在另一序列中位于两个相邻集合。遍历语料库时，每当词元对共同出现，便累加其权重。

更新：使用动作—中间节点更新语料库。每轮迭代的下一步是合并累积共现权重最高的词元对。由于词元合并可能改变集合大小，我们使用双向链表 (Zouhar et al., 2023) 维护每个动作序列，其中每个节点表示一个词元集合。合并同一集合内的词元很直接，即用新词元替换两个原词元。但合并来自相邻集合的词元更为复杂，e.g., 需要确定新词元应归入哪个集合。

中间节点。我们引入“中间节点”来处理组合多个集合特征的词元。初始时，链表中的所有节点都包含其对应动作的特定特征，这些节点称为“动作节点”。

(1) 合并来自两个相邻动作节点的词元时，在二者之间插入一个新的中间节点。新词元存入中间节点，被合并词元则从各自动作节点移除；

(2) 合并动作节点与中间节点中的词元时，新词元替换中间节点中的原词元，因为新词元同样组合了多个动作的特征。合并后，从动作节点移除相应词元。

遵循上述更新规则可保证任意两个动作节点之间至多存在一个中间节点，且每个中间节点至多包含一个词元。计算涉及中间节点的共现权重时，可将其简单视为大小为 1 的集合。

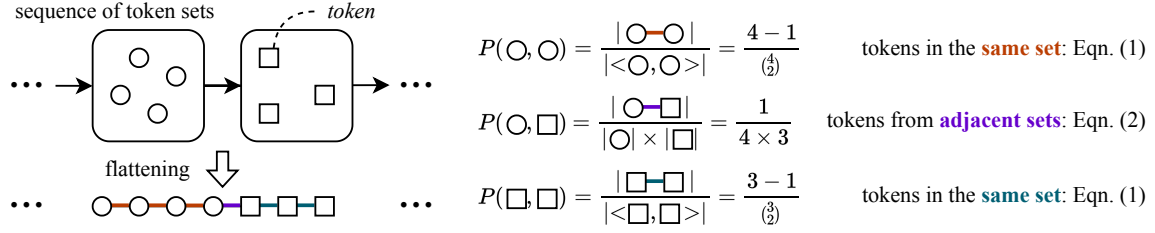


Figure 2. 词表构建期间共现词元对权重的统计方式。示例考虑序列中两个相邻集合：一个包含 4 个词元（以 $\bigcirc$ 表示），另一个包含 3 个词元（以 $\square$ 表示）。统计单个集合内部 ($\langle \bigcirc, \bigcirc \rangle$ 与 $\langle \square, \square \rangle$) 及两个相邻集合之间 ($\langle \bigcirc, \square \rangle$) 的词元对。

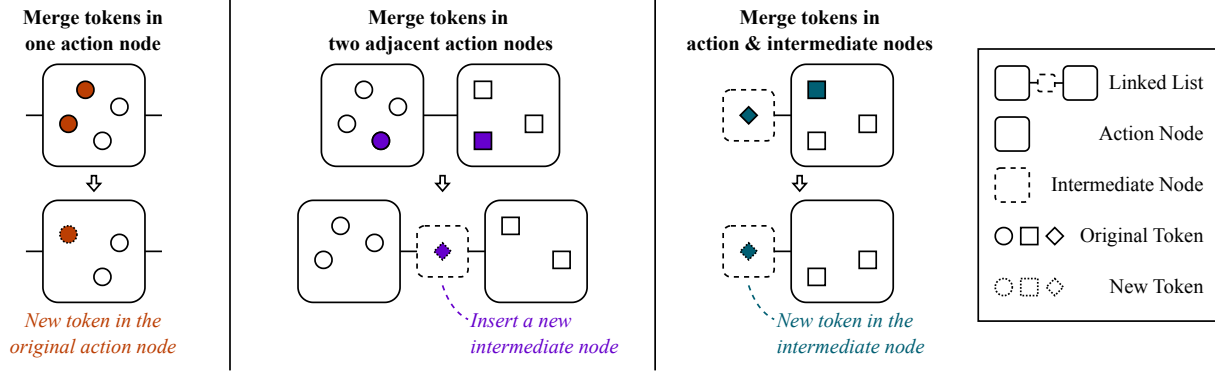


Figure 3. 将两个词元合并为新词元时，维护动作序列的链表如何更新。考虑三种情况：（1）两个词元位于同一动作节点；（2）两个词元位于相邻动作节点；（3）一个词元位于动作节点，另一个位于中间节点。

高效实现。朴素地统计并更新语料库需要 $O(QNLm^2)$ 的总时间复杂度，其中 Q 为目标词表大小， N 为训练语料库中的动作序列数， L 为这些序列的平均长度。但无需在每轮迭代中从头统计共现，因为相较上一轮，维护的链表仅有一小部分发生变化。

数据结构。为此，我们构建倒排索引，将词元对映射至包含它们的所有链表；同时维护全局堆，以返回累积共现最高的词元对。关键挑战在于更新这些数据结构。我们精确计算累积共现的变化并更新倒排索引；对堆则采用惰性更新策略，将带标签的最新权重插入堆中。取值时检查标签以确认数值是否为最新；若不是，则丢弃该值并获取下一个。

时间复杂度。令 $H = O(NLm)$ 表示堆的最大大小。所提出算法将原始时间复杂度成功降低至 $O(\log Q \log H \cdot NLm^2)$ ，实现高效词表构建。实际中，后期迭代耗时显著少于初期，这是因为累积共现权重较高的词元通常在早期频繁出现。摊还复杂度的降低缩短了总体构建时间。词表构建算法的更多细节见 Section C。

3.2.2. 基于集合置换正则化的分词

分词将原始动作序列转换为特征集合序列。分词后序列中的每个集合对应词表中的一个词元。

朴素分词。ActionPiece 的一种分词策略是沿用构建词表的技术，即迭代识别高优先级词元对（以词元 ID 表示，越早添加的词元优先级可能越高）。然而，我们观察到该策略会产生偏差，使词表中仅有部分词元被频繁使用（实证结果见 Section 4.4.2）。

集合置换正则化 (Set Permutation Regularization, SPR)。为解决上述问题并考虑集合的无序性，我们提出集合置换正则化，为每个动作序列生成多种分词结果。其核心思想是不枚举同一集合或相邻集合中词元之间的所有可能配对，而是对每个集合生成随机排列并将其视为一维序列。拼接所有排列即可得到一个长词元序列，再使用传统 BPE 分词方法 (Sennrich et al., 2016) 对其切分。不同排列可产生语义相同但分词不同的词元序列。这些序列可作为模型训练的自然增强 (Section 3.3.1)，并在模型推理时实现内生集成 (Section 3.3.2)。

Table 2. 广泛使用的文本词元化方法字节对编码 (BPE) (Sennrich et al., 2016) 与所提出的上下文感知动作序列词元化方法 ActionPiece 的对比。

方面	BPE	ActionPiece
数据类型	文本序列	动作（无序特征集合）序列
词元	字节序列	特征集合
初始词表	单字节	单特征集合
合并单元	相邻字节对	同一集合内部或相邻集合之间的特征对
共现加权	原始频次统计	概率加权 (Figure 2)
分词策略	贪心固定顺序合并	集合置换正则化 (Algorithm 4)
中间结构	不适用	用于跨动作合并的中间节点

3.3. 生成式推荐模型

3.3.1. 在增强词元序列上训练

对于训练语料库中的动作序列及其真实下一动作，我们分别将其词元化为词元序列 C_{in} 与 C_{out} 。以 C_{in} 为输入，训练 Transformer 编码器—解码器模块 (Raffel et al., 2020) 自回归生成 C_{out} (e.g., 下一词元预测目标 (Rajput et al., 2023))。训练期间，每轮都使用 Section 3.2.2 所述集合置换正则化对动作序列进行词元化，从而自然增强训练序列，并在实证中提升模型性能，见 Section 4.3。

3.3.2. 推理时集成

模型推理时，使用集合置换正则化将每个动作序列词元化 q 次。将这 q 个词元化序列输入模型，得到 q 个输出排序列表 (e.g., 以 TIGER (Rajput et al., 2023) 为 GR 骨干时使用束搜索推理)。随后对每个预测物品的得分取平均，从而合并这些排序列表。该数据级集成方法可提升推荐性能，见 Section 4.4.3。

3.4. 讨论

动作序列中的顺序。 推荐系统通常将用户动作表示为特征集合，例如关联物品的标题与价格。这些特征在单个动作内部通常没有固有顺序；但在序列推荐任务中，用户历史动作通常按时间戳排序，以捕获时间行为动态。基于此，我们将动作序列建模为特征集合序列：保持每个动作内部特征无序，同时保留动作的时间顺序。随时间演化的特征组合能够捕获有意义的序列模式。

Table 3. 处理后数据集的统计信息。“平均 t ” 表示动作序列中的平均动作数。

数据集	用户数	物品数	动作数	平均 t
Sports	35,598	18,357	260,739	8.32
Beauty	22,363	12,101	176,139	8.87
CDs	75,258	64,443	1,022,334	14.58

ActionPiece 与 BPE。 ActionPiece 虽遵循与 BPE 类似的算法框架，但关键区别在于二者建模的数据格式。BPE 作用于二维字节序列，而 ActionPiece 专为词元化特征集合序列设计。将每个动作建模为无序集合更符合动作序列的内在结构。主要差异汇总于 Table 2。

SPR 对效率的影响。 尽管引入集合置换正则化，训练效率仍与 TIGER 等现有方法相当。特征置换在 CPU 上执行，并与基于 TPU/GPU 的模型更新异步运行，因此训练速度没有明显下降。推理时，由于需要集成增强测试样本，SPR 会引入额外 FLOPs；但增强版本可在多个计算设备 (e.g., TPU 或 GPU) 上并行处理，因此总体延迟仍与基线方法相当。并行性抵消了新增计算，使本方法即使采用推理时集成，仍能保持高效推理。

4. 实验

4.1. 实验设置

数据集。 实验使用 Amazon Reviews 数据集 (McAuley et al., 2015) 的三个类别：“Sports

Table 4. 不同方法在 Amazon Reviews 数据集 (McAuley et al., 2015) 上的性能对比。最佳与次佳性能分别以**粗体**和下划线标示。“R@K”和“N@K”分别为“Recall@K”和“NDCG@K”的缩写。“提升”表示本文方法相对最强基线的百分比提升。

数据集	指标	基于 ID		特征 + ID			生成式						提升
		BERT4Rec	SASRec	FDSA	S ³ -Rec	VQ-Rec	P5-CID	TIGER	LMIndexer	HSTU	SPM-SID	ActionPiece	
Sports	R@5	0.0115	0.0233	0.0182	0.0251	0.0181	0.0287	0.0264	0.0222	0.0258	<u>0.0280</u>	0.0316 $\pm$ 0.0005	+12.86%
	N@5	0.0075	0.0154	0.0122	0.0161	0.0132	0.0179	<u>0.0181</u>	0.0142	0.0165	0.0180	0.0205 $\pm$ 0.0002	+11.71%
	R@10	0.0191	0.0350	0.0288	0.0385	0.0251	0.0426	0.0400	—	0.0414	<u>0.0446</u>	0.0500 $\pm$ 0.0007	+12.11%
	N@10	0.0099	0.0192	0.0156	0.0204	0.0154	0.0224	0.0225	—	0.0215	<u>0.0234</u>	0.0264 $\pm$ 0.0003	+12.82%
Beauty	R@5	0.0203	0.0387	0.0267	0.0387	0.0434	0.0468	0.0454	0.0415	0.0469	<u>0.0475</u>	0.0511 $\pm$ 0.0014	+7.58%
	N@5	0.0124	0.0249	0.0163	0.0244	0.0311	0.0315	0.0321	0.0262	0.0314	<u>0.0321</u>	0.0340 $\pm$ 0.0011	+5.92%
	R@10	0.0347	0.0605	0.0407	0.0647	<u>0.0741</u>	0.0701	0.0648	—	0.0704	0.0714	0.0775 $\pm$ 0.0017	+4.59%
	N@10	0.0170	0.0318	0.0208	0.0327	0.0372	<u>0.0400</u>	0.0384	—	0.0389	0.0399	0.0424 $\pm$ 0.0011	+6.00%
CDs	R@5	0.0326	0.0351	0.0226	0.0213	0.0314	0.0505	0.0492	—	0.0417	<u>0.0509</u>	0.0544 $\pm$ 0.0005	+6.88%
	N@5	0.0201	0.0177	0.0137	0.0130	0.0209	0.0326	0.0329	—	0.0275	<u>0.0337</u>	0.0359 $\pm$ 0.0004	+6.53%
	R@10	0.0547	0.0619	0.0378	0.0375	0.0485	<u>0.0785</u>	0.0748	—	0.0638	0.0778	0.0830 $\pm$ 0.0008	+5.73%
	N@10	0.0271	0.0263	0.0186	0.0182	0.0264	0.0416	0.0411	—	0.0346	<u>0.0424</u>	0.0451 $\pm$ 0.0005	+6.37%

and Outdoors” (Sports)、“Beauty” (Beauty) 和 “CDs and Vinyl” (CDs)。每位用户的历史评论被视为“动作”，并按时间顺序排列成动作序列，较早的评论在前。模型评估采用广泛使用的留一法协议 (Kang & McAuley, 2018; Zhao et al., 2022; Rajput et al., 2023)，每个动作序列的最后一个物品与倒数第二个物品分别用于测试和验证。处理后数据集的统计信息见 Table 3，更多细节见 Section F。

对比方法。我们将 ActionPiece 与以下方法比较：（1）基于 ID 的序列推荐方法，包括 BERT4Rec (Sun et al., 2019) 和 SASRec (Kang & McAuley, 2018)；（2）特征增强的序列推荐方法，如 FDSA (Zhang et al., 2019)、S³-Rec (Zhou et al., 2020) 和 VQ-Rec (Hou et al., 2023)；（3）生成式推荐方法，包括 P5-CID (Hua et al., 2023)、TIGER (Rajput et al., 2023)、LMIndexer (Jin et al., 2024)、HSTU (Zhai et al., 2024) 和 SPM-SID (Singh et al., 2024)，它们分别代表不同的动作词元化方法 (Table 1)。基线详情见 Section G。

评估设置。遵循 Rajput et al. (2023)，使用 Recall@K 与 NDCG@K 作为评估指标，其中 $K \in \{5, 10\}$ 。选取验证集表现最佳的模型检查点在测试集上评估。实验使用五个随机种子并报告平均指标。

实现细节。详细实现与超参数设置见 Section H。

4.2. 总体性能

我们在三个公开数据集上，将 ActionPiece 与采用不同动作词元化方法的序列推荐和生成式推荐基线进行比较，结果见 Table 4。

对比结果显示，使用物品特征的方法通常优于仅使用物品 ID 的方法，说明引入特征可增强推荐性能。在利用物品特征的方法（“特征 + ID”与“生成式”）中，生成式推荐模型表现更好。这进一步证实，将语义注入物品索引并在子物品层面优化，可使生成模型更充分利用语义信息并提升推荐性能。所有基线中，SPM-SID 表现最佳；它引入 SentencePiece 模型 (Kudo & Richardson, 2018)，用新词元替换每个物品内部常见的语义 ID 模式，从更大词表中获益。

所提出的 ActionPiece 在三个数据集上始终优于所有基线，NDCG@10 显著提升，较最佳基线高出 6.00% 至 12.82%。ActionPiece 是首个上下文感知动作序列词元化器，即同一动作可根据周围上下文被词元化为不同词元，从而捕获重要的序列级特征模式并提升推荐性能。

4.3. 消融实验

我们在 Table 5 中进行消融分析，以研究各项技术对 ActionPiece 的贡献。

（1）为检验 ActionPiece 的性能增益是否源于词

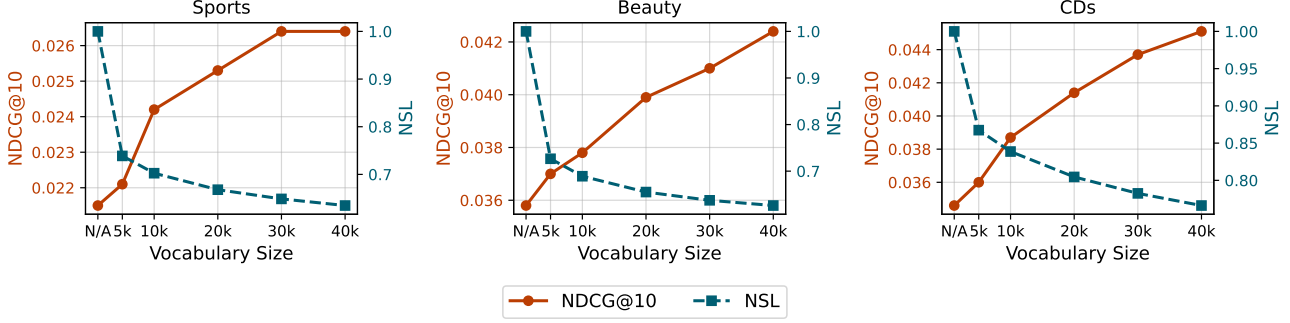


Figure 4. 三个数据集上推荐性能 (NDCG@10, $\uparrow$) 及平均词元化序列长度 (NSL, $\downarrow$) 随词表大小的变化。“N/A”表示不应用 ActionPiece, 即动作序列仅由初始词元表示。

Table 5. ActionPiece 的消融分析。推荐性能以 NDCG@10 衡量, 最佳性能以**粗体**标示。

变体	Sports	Beauty	CDs
不同词表大小的 TIGER			
(1.1) TIGER - 192 (4×48)	0.0231	0.0362	N/A [†]
(1.2) TIGER - 768 (3×2^8)	0.0220	0.0378	0.0331
(1.3) TIGER - 1k (4×2^8)	0.0225	0.0384	0.0411
(1.4) TIGER-49k (6×2^{13})	0.0162	0.0317	0.0338
(1.5) TIGER-66k (4×2^{14})	0.0194	N/A [‡]	0.0319
词表构建			
(2.1) 不使用词元化	0.0215	0.0389	0.0346
(2.2) 不使用上下文感知	0.0258	0.0416	0.0429
(2.3) 不使用加权统计	0.0257	0.0412	0.0435
集合置换正则化			
(3.1) 仅用于推理	0.0192	0.0316	0.0329
(3.2) 仅用于训练	0.0244	0.0387	0.0422
(3.3) TIGER + SPR	0.0202	0.0330	0.0351
ActionPiece (40k)	0.0264	0.0424	0.0451

[†] 不适用, 因为 CDs 中语义 ID 前缀 (前三个词元) 的冲突数超过 48。

[‡] 不适用, 因为 2^{14} 大于 Beauty 的物品数。

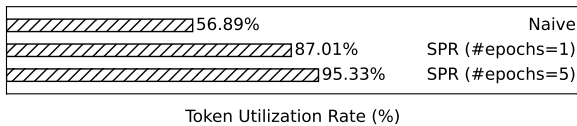


Figure 5. 模型训练期间词元利用率 (%) 随分词策略的变化。

表大小, 我们改变 TIGER 的词表大小: 增加每个物品的语义 ID 位数 ($4 \rightarrow 6$) 及每位候选语义 ID 数 ($2^8 \rightarrow 2^{13}$ 或 2^{14}), 得到两个词表大于 ActionPiece 的 TIGER 变体; 同时通过减少位数或每位

候选数构造两个较小词表变体。尽管覆盖更广的词表大小, 所有 TIGER 变体都弱于 ActionPiece, 有时甚至弱于含 1,024 个词元的原始 TIGER。这说明 ActionPiece 的提升并非简单来自词表扩缩, 也凸显了生成式推荐模型有效扩展词表的困难, 与 Zhang et al. (2024) 的观察一致。

(2) 为评估词表构建技术, 引入以下变体: (2.1) 不使用词元化, 跳过词表构建, 直接以物品特征作为词元; (2.2) 不使用上下文感知, 构建词表及分词时仅考虑每个动作内部的共现与词元合并; (2.3) 不使用加权统计, 同等对待所有词元对, 而不使用 Equations (1) and (2) 中定义的权重。移除任一技术都会降低性能, 说明它们对构建上下文感知词元化器十分重要。

(3) 为评估 SPR, 我们分别在模型训练与推理中恢复 Section 3.2.2 所述朴素分词。训练或推理中用朴素分词替代 SPR 都会降低性能。我们还将 SPR 应用于现有 GR 模型 TIGER, 结果表明单独使用 SPR 不足以改善 GR 模型。对上下文无关方法词元化的动作序列应用 SPR 时, 它不会改变词元频率, 只会打乱基于 RQ-VAE 的语义 ID 内部词元顺序。相比之下, ActionPiece 根据相邻上下文为同一物品导出不同词元, 既提高词元利用率, 又形成有效的数据增强。

4.4. 进一步分析

4.4.1. 性能与效率随词表大小的变化

词表大小是语言词元化器的关键超参数 (Meta AI, 2024; Dagan et al., 2024)。我们研究调整词表大小

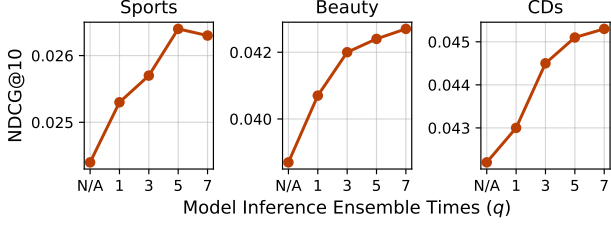


Figure 6. 模型推理期间, 性能 (NDCG@10, $\uparrow$) 随集成分词结果数 q 的变化。

对生成式推荐模型的影响, 并使用归一化序列长度 (NSL) (Dagan et al., 2024) 衡量词元化序列长度; NSL 越小, 表示每个序列的词元越少。实验词表大小为 {N/A, 5k, 10k, 20k, 30k, 40k}, “N/A” 表示直接以物品特征作为词元。如 Figure 4 所示, 增大词表可改善推荐性能并缩短词元化序列; 反之, 减小词表会减少模型参数并提高内存效率。因此, 调整词表大小可在模型性能、序列长度与内存效率之间进行权衡。

4.4.2. 词元利用率随分词策略的变化

如 Section 3.3.1 所述, SPR 通过生成多个语义相同的词元序列增强训练语料库。Table 5 显示, 引入 SPR 显著改善推荐性能, 一个可能原因是其提高了词元利用率。为验证这一假设, 我们在每轮训练中分别使用朴素分词与 SPR 切分动作序列。如 Figure 5 所示, 朴素分词仅使用 56.89% 的词元训练模型, 限制了模型泛化至未见动作序列的能力; SPR 在首轮训练后即达到 87.01% 的词元利用率, 并随训练推进进一步提高。这表明 SPR 改善了 ActionPiece 词元利用率, 从而增强泛化能力与性能。

4.4.3. 性能随推理时集成数量的变化

如 Section 3.3.2 所述, ActionPiece 使用 SPR 分词支持推理时集成。我们将集成分词结果数 q 设为 {N/A, 1, 3, 5, 7}, “N/A” 表示模型推理时使用朴素分词。如 Figure 6 所示, 集成更多词元化序列可改善 ActionPiece 的推荐性能; 但当 q 增至 5 和 7 时, 性能增益趋缓。更高的 q 也会增加推理计算成本, 因此实践中需在性能与计算预算间权衡。

4.5. 案例分析

为理解 GR 模型如何从无序特征设定与上下文感知动作序列词元化中获益, Figure 1 给出了一个示例。

动作序列中的每个物品均表示为由五个特征组成的特征集合, 物品内部的特征无需特定顺序。词元化第一步利用特征集合的无序性, 并应用集合置换正则化 (Section 3.2.2)。该过程将每个特征集合排列为某个特定置换, 再依据构建的词表 (Section 3.2.1) 迭代组合特征, 由此得到语义相同的不同分词结果。每个结果表示为集合序列, 其中每个集合对应词表中的一个词元。

通过观察分词结果及对应词元序列, 可识别 Figure 1 标注的四类词元: (1) 单个物品的部分特征 (词元 14844 对应 T 恤的特征 747 与 923); (2) 仅含一个特征的集合 (袜子的特征 76); (3) 单个物品的全部特征 (词元 7995 对应短裤的全部特征); (4) 来自多个物品的特征 (e.g., 词元 8316 包含 T 恤特征 923 与袜子特征 679, 词元 19895 则包含袜子特征 1100 及短裤特征 560、943)。第四类表明, 一个动作的特征可与相邻动作的特征共同切分、分组, 使同一动作随周围上下文不同而得到不同词元, 体现了 ActionPiece 的上下文感知词元化过程。

5. 结论

本文提出 ActionPiece, 它是首个面向生成式推荐的上下文感知动作序列词元化器。通过考虑周围上下文, 同一动作可在不同序列中被词元化为不同词元。我们将生成式推荐表述为特征集合序列上的任务, 并将重要的特征模式合并为词元。在构建词表时, 我们根据词元对的结构为其赋予权重, 例如区分同一集合内部与相邻集合之间的词元对。为高效构建词表, 我们使用双向链表维护语料库, 并引入中间节点来存储由相邻集合间特征组合而成的词元。此外, 我们提出集合置换正则化, 将单个动作序列切分为多个语义相同的词元序列。这些分词结果既可作为训练时的自然数据增强, 也可作为推理时的集成实例。

未来，我们计划通过构造融合 ActionPiece 词元与其他类型词元的指令，将用户动作与其他模态对齐。我们还计划将所提出的词元化器扩展到其他可表述为集合序列建模问题的任务，包括音频建模、序列决策和时间序列预测。

致谢

本工作于 Yupeng Hou 在 Google DeepMind 担任学生研究员期间完成。感谢 Yichen Zhou 的有益讨论，以及 Nikhil Mehta 对论文草稿提出的宝贵建议。

影响声明

本文提出 ActionPiece，一种用于增强生成式推荐的上下文感知动作序列词元化器。本工作旨在推动个性化推荐系统发展，使其能够更准确地理解用户偏好。我们认为，本工作与特定社会或伦理问题不存在直接关联，其影响主要取决于推荐系统在电子商务、娱乐和社交平台等领域中的广泛应用及其潜在影响。

参考文献

- Anil, R., Borgeaud, S., Wu, Y., Alayrac, J., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., Silver, D., Petrov, S., Johnson, M., Antonoglou, I., Schrittwieser, J., Glaese, A., Chen, J., Pitler, E., Lillicrap, T. P., Lazaridou, A., Firat, O., Molloy, J., Isard, M., Barham, P. R., Hennigan, T., Lee, B., Viola, F., Reynolds, M., Xu, Y., Doherty, R., Collins, E., Meyer, C., Rutherford, E., Moreira, E., Ayoub, K., Goel, M., Tucker, G., Piqueras, E., Krikun, M., Barr, I., Savinov, N., Danihelka, I., Roelofs, B., White, A., Andreassen, A., von Glehn, T., Yagati, L., Kazemi, M., Gonzalez, L., Khalman, M., Sygnowski, J., and et al. Gemini: A family of highly capable multimodal models. *arxiv:2312.11805*, 2023.
- Bahdanau, D., Cho, K., and Bengio, Y. Neural machine translation by jointly learning to align and translate. In *ICLR*, 2015.
- Bao, K., Zhang, J., Zhang, Y., Wang, W., Feng, F., and He, X. TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In *RecSys*, 2023.
- Bian, W., Wu, K., Ren, L., Pi, Q., Zhang, Y., Xiao, C., Sheng, X.-R., Zhu, Y.-N., Chan, Z., Mou, N., et al. CAN: feature co-action network for click-through rate prediction. In *WSDM*, pp. 57–65, 2022.
- Cheng, H.-T., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhya, H., Anderson, G., Corrado, G., Chai, W., Ispir, M., Anil, R., Haque, Z., Hong, L., Jain, V., Liu, X., and Shah, H. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pp. 7–10, 2016.
- Dagan, G., Synnaeve, G., and Roziere, B. Getting the most out of your tokenizer for pre-training and domain adaptation. In *ICML*, 2024.
- Deldjoo, Y., He, Z., McAuley, J., Korikov, A., Saner, S., Ramisa, A., Vidal, R., Sathiamoorthy, M., Kasirzadeh, A., and Milano, S. A review of modern recommender systems using generative models (gen-recsys). In *KDD*, pp. 6448–6458, 2024.
- Ding, Y., Hou, Y., Li, J., and McAuley, J. Inductive generative recommendation via retrieval-based speculation. *arXiv preprint arXiv:2410.02939*, 2024.
- Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library. *arXiv preprint arXiv:2401.08281*, 2024.
- Ge, T., He, K., Ke, Q., and Sun, J. Optimized product quantization for approximate nearest neighbor search. In *CVPR*, pp. 2946–2953, 2013.
- Geng, S., Liu, S., Fu, Z., Ge, Y., and Zhang, Y. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *RecSys*, pp. 299–315, 2022.
- He, X., Haffari, G., and Norouzi, M. Dynamic programming encoding for subword segmentation in neural machine translation. In *ACL*, pp. 3042–3051, 2020.
- Hidasi, B., Karatzoglou, A., Baltrunas, L., and Tikk, D. Session-based recommendations with recurrent neural networks. In *ICLR*, 2016.
- Hofmann, V., Schuetze, H., and Pierrehumbert, J. An embarrassingly simple method to mitigate undesirable properties of pretrained language model tokenizers. In *ACL*, 2022.

- Hou, Y., Mu, S., Zhao, W. X., Li, Y., Ding, B., and Wen, J.-R. Towards universal sequence representation learning for recommender systems. In KDD, pp. 585–593, 2022.
- Hou, Y., He, Z., McAuley, J., and Zhao, W. X. Learning vector-quantized item representation for transferable sequential recommenders. In WWW, pp. 1162–1171, 2023.
- Hou, Y., Li, J., He, Z., Yan, A., Chen, X., and McAuley, J. Bridging language and items for retrieval and recommendation. arXiv preprint arXiv:2403.03952, 2024a.
- Hou, Y., Zhang, J., Lin, Z., Lu, H., Xie, R., McAuley, J., and Zhao, W. X. Large language models are zero-shot rankers for recommender systems. In ECIR, 2024b.
- Hou, Y., Zhang, A., Sheng, L., Yang, Z., Wang, X., Chua, T.-S., and McAuley, J. Generative recommendation models: Progress and directions. In Companion Proceedings of the ACM on Web Conference 2025, pp. 13–16, 2025.
- Hua, W., Xu, S., Ge, Y., and Zhang, Y. How to index item ids for recommendation foundation models. In SIGIR-AP, pp. 195–204, 2023.
- Jin, B., Zeng, H., Wang, G., Chen, X., Wei, T., Li, R., Wang, Z., Li, Z., Li, Y., Lu, H., Wang, S., Han, J., and Tang, X. Language models as semantic indexers. In ICML, 2024.
- Juan, Y., Zhuang, Y., Chin, W.-S., and Lin, C.-J. Field-aware factorization machines for ctr prediction. In RecSys, pp. 43–50, 2016.
- Kang, W.-C. and McAuley, J. Self-attentive sequential recommendation. In ICDM, pp. 197–206, 2018.
- Kim, S., Kang, H., Choi, S., Kim, D., Yang, M., and Park, C. Large language models meet collaborative filtering: An efficient all-round llm-based recommender system. In KDD, pp. 1395–1406, 2024.
- Kudo, T. Subword regularization: Improving neural network translation models with multiple subword candidates. In ACL, pp. 66–75, 2018.
- Kudo, T. and Richardson, J. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In EMNLP, 2018.
- Li, G., Zhang, X., Zhang, Y., Yin, Y., Yin, G., and Lin, W. Semantic convergence: Harmonizing recommender systems via two-stage alignment and behavioral semantic tokenization. In AAAI, 2025.
- Li, J., Ren, P., Chen, Z., Ren, Z., Lian, T., and Ma, J. Neural attentive session-based recommendation. In CIKM, pp. 1419–1428, 2017.
- Liao, J., Li, S., Yang, Z., Wu, J., Yuan, Y., Wang, X., and He, X. LLaRA: Large language-recommendation assistant. In SIGIR, 2024.
- Liu, E., Zheng, B., Ling, C., Hu, L., Li, H., and Zhao, W. X. End-to-end learnable item tokenization for generative recommendation. In SIGIR, 2025.
- Liu, H., Wei, Y., Song, X., Guan, W., Li, Y.-F., and Nie, L. Mmgrec: Multimodal generative recommendation with transformer model. arXiv preprint arXiv:2404.16555, 2024a.
- Liu, Z., Hou, Y., and McAuley, J. Multi-behavior generative recommendation. In CIKM, 2024b.
- McAuley, J., Targett, C., Shi, Q., and Van Den Hengel, A. Image-based recommendations on styles and substitutes. In SIGIR, pp. 43–52, 2015.

- Meta AI. Introducing Meta Llama 3: The most capable openly available LLM to date, 2024. URL <https://ai.meta.com/blog/meta-llama-3/>.
- Ni, J., Abrego, G. H., Constant, N., Ma, J., Hall, K., Cer, D., and Yang, Y. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. In *Findings of ACL*, pp. 1864–1874, 2022.
- OpenAI. Introducing ChatGPT, 2022. URL <https://openai.com/index/chatgpt/>.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. *NeurIPS*, 32, 2019.
- Pazzani, M. J. and Billsus, D. Content-based recommendation systems. In *The adaptive web: methods and strategies of web personalization*, pp. 325–341. Springer, 2007.
- Petrov, A. V. and Macdonald, C. Generative sequential recommendation with gptrec. *arXiv preprint arXiv:2306.11114*, 2023.
- Provilkov, I., Emelianenko, D., and Voita, E. Bpe-dropout: Simple and effective subword regularization. In *ACL*, pp. 1882–1892, 2020.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.
- Rajput, S., Mehta, N., Singh, A., Keshavan, R. H., Vu, T., Heldt, L., Hong, L., Tay, Y., Tran, V. Q., Samost, J., Kula, M., Chi, E. H., and Sathiamoorthy, M. Recommender systems with generative retrieval. In *NeurIPS*, 2023.
- Schmidt, C. W., Reddy, V., Zhang, H., Alameddine, A., Uzan, O., Pinter, Y., and Tanner, C. Tokenization is more than compression. In *EMNLP*, 2024.
- Sennrich, R., Haddow, B., and Birch, A. Neural machine translation of rare words with subword units. In *ACL*, pp. 1715–1725, 2016.
- Shan, Y., Hoens, T. R., Jiao, J., Wang, H., Yu, D., and Mao, J. Deep crossing: Web-scale modeling without manually crafted combinatorial features. In *KDD*, pp. 255–262, 2016.
- Sheng, L., Zhang, A., Zhang, Y., Chen, Y., Wang, X., and Chua, T.-S. Language representations can be what recommenders need: Findings and potentials. In *ICLR*, 2025.
- Si, Z., Sun, Z., Chen, J., Chen, G., Zang, X., Zheng, K., Song, Y., Zhang, X., Xu, J., and Gai, K. Generative retrieval with semantic tree-structured item identifiers via contrastive learning. In *SIGIR-AP*, 2024.
- Singh, A., Vu, T., Mehta, N., Keshavan, R., Sathiamoorthy, M., Zheng, Y., Hong, L., Heldt, L., Wei, L., Tandon, D., Chi, E. H., and Yi, X. Better generalization with semantic ids: A case study in ranking for recommendations. In *RecSys*, 2024.
- Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., and Jiang, P. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *CIKM*, pp. 1441–1450, 2019.
- Sutskever, I., Vinyals, O., and Le, Q. V. Sequence to sequence learning with neural networks. In *NIPS*, 2014.
- Tan, J., Xu, S., Hua, W., Ge, Y., Li, Z., and Zhang, Y. Idgenrec: Llm-recsys alignment with textual id learning. In *SIGIR*, 2024.

- Tay, Y., Tran, V., Dehghani, M., Ni, J., Bahri, D., Mehta, H., Qin, Z., Hui, K., Zhao, Z., Gupta, J. P., Schuster, T., Cohen, W. W., and Metzler, D. Transformer memory as a differentiable search index. In *NeurIPS*, 2022.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models. *arXiv:2302.13971*, 2023.
- Wang, R., Fu, B., Fu, G., and Wang, M. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD’17*, pp. 1–7. 2017.
- Wang, R., Shivanna, R., Cheng, D., Jain, S., Lin, D., Hong, L., and Chi, E. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *WWW*, pp. 1785–1797, 2021.
- Wang, W., Bao, H., Lin, X., Zhang, J., Li, Y., Feng, F., Ng, S.-K., and Chua, T.-S. Learnable tokenizer for llm-based generative recommendation. In *CIKM*, 2024a.
- Wang, Y., Ren, Z., Sun, W., Yang, J., Liang, Z., Chen, X., Xie, R., Yan, S., Zhang, X., Ren, P., Chen, Z., and Xin, X. Content-based collaborative generation for recommender systems. In *CIKM*, 2024b.
- Wang, Y., Xun, J., Hong, M., Zhu, J., Jin, T., Lin, W., Li, H., Li, L., Xia, Y., Zhao, Z., and Dong, Z. Eager: Two-stream generative recommender with behavior-semantic collaboration. In *KDD*, pp. 3245–3254, 2024c.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. Transformers: State-of-the-art natural language processing. In *EMNLP*, 2020.
- Wu, Y., Schuster, M., Chen, Z., Le, Q. V., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K., Klingner, J., Shah, A., Johnson, M., Liu, X., Łukasz Kaiser, Gouws, S., Kato, Y., Kudo, T., Kazawa, H., Stevens, K., Kurian, G., Patil, N., Wang, W., Young, C., Smith, J., Riesa, J., Rudnick, A., Vinyals, O., Corrado, G., Hughes, M., and Dean, J. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- Yang, L., Paischer, F., Hassani, K., Li, J., Shao, S., Li, Z. G., He, Y., Feng, X., Noorshams, N., Park, S., Long, B., Nowak, R. D., Gao, X., and Eghbalzadeh, H. Unifying generative and dense retrieval for sequential recommendation. *arXiv preprint arXiv:2411.18814*, 2024.
- Yehezkel, S. and Pinter, Y. Incorporating context into subword vocabularies. In *EACL*, pp. 623–635, 2023.
- Yuan, Z., Yuan, F., Song, Y., Li, Y., Fu, J., Yang, F., Pan, Y., and Ni, Y. Where to go next for recommender systems? id-vs. modality-based recommender models revisited. In *SIGIR*, pp. 2639–2649, 2023.
- Zeghidour, N., Luebs, A., Omran, A., Skoglund, J., and Tagliasacchi, M. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2021.
- Zhai, J., Liao, L., Liu, X., Wang, Y., Li, R., Cao, X., Gao, L., Gong, Z., Gu, F., He, M., Lu, Y., and

- Shi, Y. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. In ICML, 2024.
- Zhang, J., Xie, R., Hou, Y., Zhao, W. X., Lin, L., and Wen, J.-R. Recommendation as instruction following: A large language model empowered recommendation approach. *ACM Trans. Inf. Syst.*, 2025a.
- Zhang, T., Zhao, P., Liu, Y., Sheng, V. S., Xu, J., Wang, D., Liu, G., and Zhou, X. Feature-level deeper self-attention network for sequential recommendation. In IJCAI, pp. 4320–4326, 2019.
- Zhang, T., Pan, J., Wang, J., Zha, Y., Dai, T., Chen, B., Luo, R., Deng, X., Wang, Y., Yue, M., et al. Towards scalable semantic representation for recommendation. *arXiv preprint arXiv:2410.09560*, 2024.
- Zhang, Y., Feng, F., Zhang, J., Bao, K., Wang, Q., and He, X. CoLLM: Integrating collaborative embeddings into large language models for recommendation. *IEEE Transactions on Knowledge and Data Engineering*, 37(5), 2025b.
- Zhao, W. X., Mu, S., Hou, Y., Lin, Z., Chen, Y., Pan, X., Li, K., Lu, Y., Wang, H., Tian, C., Min, Y., Feng, Z., Fan, X., Chen, X., Wang, P., Ji, W., Li, Y., Wang, X., and Wen, J.-R. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In CIKM, 2021.
- Zhao, W. X., Lin, Z., Feng, Z., Wang, P., and Wen, J.-R. A revisiting study of appropriate offline evaluation for top-n recommendation algorithms. *ACM Transactions on Information Systems*, 41(2):1–41, 2022.
- Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J., and Wen, J. A survey of large language models. *arXiv:2303.18223*, 2023.
- Zheng, B., Hou, Y., Lu, H., Chen, Y., Zhao, W. X., Chen, M., and Wen, J.-R. Adapting large language models by integrating collaborative semantics for recommendation. In ICDE, pp. 1435–1448, 2024.
- Zheng, B., Zhang, J., Lu, H., Chen, Y., Chen, M., Zhao, W. X., and Wen, J.-R. Enhancing graph contrastive learning with reliable and informative augmentation for recommendation. In KDD, 2025.
- Zhou, K., Wang, H., Zhao, W. X., Zhu, Y., Wang, S., Zhang, F., Wang, Z., and Wen, J.-R. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In CIKM, pp. 1893–1902, 2020.
- Zhu, J., Jin, M., Liu, Q., Qiu, Z., Dong, Z., and Li, X. Cost: Contrastive quantization based semantic tokenization for generative recommendation. In RecSys, 2024a.
- Zhu, Y., Wu, L., Guo, Q., Hong, L., and Li, J. Collaborative large language model for recommender systems. In WWW, pp. 3162–3172, 2024b.
- Zouhar, V., Meister, C., Gastaldi, J., Du, L., Vieira, T., Sachan, M., and Cotterell, R. A formal perspective on byte-pair encoding. In Findings of ACL, pp. 598–614, 2023.

Table 6. 符号及说明。

符号	说明
i, i_1, i_j	物品、物品标识符、物品 ID
t	输入动作序列中的动作数；模型作出预测时的时间戳
i_{t+1}	真实下一物品
$\hat{i}_{t+1}$	预测的下一物品
$S = \{i_1, i_2, \dots, i_t\}$	每个动作以交互物品 ID 表示的动作序列
$\mathcal{A}, \mathcal{A}_1, \mathcal{A}_j$	物品特征或词元的集合
$m = \mathcal{A}_j $	每个物品关联的特征数
$f_{j,k}$	物品 i_j 的第 k 个特征
$\mathcal{F}_k$	第 k 个特征所有可能取值的集合
$S' = \{\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_t\}$	每个动作以物品特征集合表示的动作序列
c, c_1, c_j	输入与生成词元
l	词元序列中的词元数
$C = \{c_1, c_2, \dots, c_l\}$	由输入动作序列 S' 词元化得到的词元序列
$\{c_{l+1}, \dots, c_q\}$	GR 模型生成的词元
$\mathcal{V}$	ActionPiece 词元化器的词表
$\mathcal{R}$	ActionPiece 词元化器的合并规则
$\{(c_u, c_v) \rightarrow c_{\text{new}}\}$	一条合并规则，表示相邻词元 c_u 与 c_v 可由 c_{new} 替换
$Q = \mathcal{V} $	ActionPiece 词表大小
$P(c, c')$	将集合序列展平为词元序列时，词元 c 与 c' 相邻的概率
N	训练语料库中的动作序列数
L	训练语料库中动作序列的平均长度
H	最大堆大小， $O(NLm)$
q	推理期间使用集合置换正则化生成的分词结果数

附录

A. 符号

本文所用符号汇总于 Table 6。

B. 算法细节

本节给出词表构建与分词的详细算法。

Algorithm 2 ActionPiece 词表构建——统计 (Figure 2)

input 动作序列语料库 $\mathcal{S}'$, 当前词表 $\mathcal{V}$ output 累积加权词元共现 $\text{count}(\cdot, \cdot)$

```

1: for  $i \leftarrow 0$  to  $|\mathcal{V}|, j \leftarrow 0$  to  $|\mathcal{V}|$  do
2:    $\text{count}(c_i, c_j) \leftarrow 0$ 
3: end for
4: for all 序列  $S' \in \mathcal{S}'$  do
5:    $t \leftarrow \text{length}(S')$  # 序列中的动作节点数
6:   for  $k \leftarrow 0$  to  $t - 1$  do
7:      $\mathcal{A}_k \leftarrow S'[k]$  # 当前动作节点
8:     # 处理  $\mathcal{A}_k$  内所有无序词元对
9:     for all  $c_i, c_j \in \mathcal{A}_k, i \neq j$  do
10:       $\text{count}(c_i, c_j) \leftarrow \text{count}(c_i, c_j) + 2/|\mathcal{A}_k|$  # 同一集合内词元权重 (Equation (1))
11:       $\text{count}(c_j, c_i) \leftarrow \text{count}(c_j, c_i) + 2/|\mathcal{A}_k|$  # 对称更新
12:    end for
13:    # 处理  $\mathcal{A}_k$  与  $\mathcal{A}_{k+1}$  间所有有序词元对
14:    if  $k < t - 1$  then
15:       $\mathcal{A}_{k+1} \leftarrow S'[k + 1]$ 
16:      for all  $c_i \in \mathcal{A}_k, c_j \in \mathcal{A}_{k+1}$  do
17:         $\text{count}(c_i, c_j) \leftarrow \text{count}(c_i, c_j) + 1/(|\mathcal{A}_k| \times |\mathcal{A}_{k+1}|)$  # 两个相邻集合中的词元权重 (Equation (2))
18:      end for
19:    end if
20:  end for
21: end for
返回  $\text{count}(\cdot, \cdot)$ 

```

B.1. 词表构建算法

词表构建的总体流程见 Algorithm 1。如 Section 3.2.1 所述, 该过程迭代执行**统计** (Algorithm 2) 与**更新** (Algorithm 3) 操作。

B.2. 集合置换正则化分词算法

使用集合置换正则化 (SPR) 将动作序列切分为词元序列的详细算法见 Algorithm 4。实践中, 我们通常多次运行 Algorithm 4, 以增强训练语料库或集成推荐输出, 详见 Sections 3.3.1 and 3.3.2。

C. 高效词表构建实现

为高效构建 ActionPiece 词表, 我们使用带惰性更新的堆、链表与倒排索引等数据结构, 加速构建过程的每轮迭代。核心思想是不在每轮重新计算词元共现, 而是更新数据结构。伪代码见 Figure 7。

Algorithm 3 ActionPiece 词表构建——更新 (Figure 3)

input 更新前的动作序列语料库 $\mathcal{S}'$, 当前合并规则 $\{(c_u, c_v) \rightarrow c_{\text{new}}\}$ output 更新后的动作序列语料库 $\mathcal{S}'$

```

1: for all 序列  $S' \in \mathcal{S}'$  do
2:    $t \leftarrow \text{length}(S')$ 
3:   for  $k \leftarrow 0$  to  $t - 1$  do
4:      $\mathcal{A}_k \leftarrow S'[k]$ 
5:     # 合并同一动作节点中的词元
6:     if  $c_u \in \mathcal{A}_k$  and  $c_v \in \mathcal{A}_k$  then
7:       用  $c_{\text{new}}$  替换  $\mathcal{A}_k$  中的  $c_u$  与  $c_v$ 
8:     end if
9:     # 合并两个相邻节点中的词元
10:    if  $k < t - 1$  then
11:       $\mathcal{A}_{k+1} \leftarrow S'[k + 1]$ 
12:      if  $c_u \in \mathcal{A}_k$  and  $c_v \in \mathcal{A}_{k+1}$  then
13:        if  $\mathcal{A}_k, \mathcal{A}_{k+1}$  均为动作节点 then
14:          在  $\mathcal{A}_k$  与  $\mathcal{A}_{k+1}$  之间创建中间节点  $M$ 
15:           $M \leftarrow \{c_{\text{new}}\}$  # Linked list:  $\mathcal{A}_k \rightarrow M \rightarrow \mathcal{A}_{k+1}$ 
16:           $\mathcal{A}_k \leftarrow \mathcal{A}_k \setminus c_u$ 
17:           $\mathcal{A}_{k+1} \leftarrow \mathcal{A}_{k+1} \setminus c_v$ 
18:        else if  $\mathcal{A}_k$  为中间节点 then
19:           $\mathcal{A}_k \leftarrow \{c_{\text{new}}\}$ 
20:           $\mathcal{A}_{k+1} \leftarrow \mathcal{A}_{k+1} \setminus c_v$ 
21:        else if  $\mathcal{A}_{k+1}$  为中间节点 then
22:           $\mathcal{A}_k \leftarrow \mathcal{A}_k \setminus c_u$ 
23:           $\mathcal{A}_{k+1} \leftarrow \{c_{\text{new}}\}$ 
24:        end if
25:      end if
26:    end if
27:  end for
28: end for
返回  $\mathcal{S}'$ 

```

C.1. 数据结构

所提出算法的数据结构经专门设计，以优化词表构建效率，其作用与实现如下：

- **链表：**训练语料库中的每个动作序列均存为链表，可在词元合并期间进行高效局部更新。当词元对 (c_u, c_v) 被新词元 c_{new} 替换时，只需修改链表中受影响节点及其邻居（见 Algorithm 3 and Figure 3）。
- **带惰性更新的堆：**最大堆按共现次数确定词元对优先级。每轮无需完全重算堆，而是采用“惰性更新”：

Algorithm 4 基于集合置换正则化 (SPR) 的分词 (Section 3.2.2)

input 动作序列 S , 合并规则 $\mathcal{R}$ output 分词后的词元序列 C

```

1:  $C \leftarrow []$  # 初始化置换后的初始词元序列
2: for all 词元集合  $\mathcal{A}_i \in S$  do
3:   将  $\mathcal{A}_i$  随机置换为  $[c_1, c_2, \dots, c_{|\mathcal{A}_i|}]$ 
4:   用  $[c_1, c_2, \dots, c_{|\mathcal{A}_i|}]$  扩展  $C$  # 拼接各置换
5: end for
6:
7: # 对置换序列应用 BPE (Sennrich et al., 2016) 分词
8: repeat
9:    $\mathcal{R}' \leftarrow \emptyset$  # 候选合并规则
10:  for  $i \leftarrow 0$  to  $|C| - 1$  do
11:    if  $\{(c_i, c_{i+1}) \rightarrow c'\} \in \mathcal{R}$  then
12:       $\mathcal{R}' \leftarrow \mathcal{R}' \cup \{(c_i, c_{i+1}) \rightarrow c'\}$ 
13:    end if
14:  end for
15:  选择  $\mathcal{R}'$  中在全部合并规则  $\mathcal{R}$  里索引最小的  $\{(c_k, c_{k+1}) \rightarrow c'\}$ 
16:   $C \leftarrow [c_1, \dots, c_{k-1}, c', c_{k+2}, \dots]$  # 用新词元  $c'$  替换  $(c_k, c_{k+1})$ 
17: until  $\mathcal{R}'$  is  $\emptyset$ 
返回  $C$ 

```

保留过期条目，但提取时跳过。伪代码通过 `is_outdated` 检查堆顶是否过期，丢弃无效条目，只处理有效条目；更新后的共现以新条目压入堆中（使用负计数模拟最大堆）。

- **倒排索引:** `pair2head` 字典将词元对映射到包含它们的序列。合并 (c_u, c_v) 时，算法通过 `pair2head[(c_u, c_v)]` 直接取得受影响的序列 ID，避免扫描全部语料库。合并后增量更新索引：将新词元对（e.g., (c_{prev}, c_{new}) 与 (c_{new}, c_{next}) ）加入 `pair2head`，并移除过时词元对，从而实现定向更新并高效访问相关序列。

这些结构将计算集中于语料库中动态变化的部分，避免冗余全局操作，从而共同降低时间复杂度。链表支持局部编辑，堆减少优先级重算，倒排索引消除暴力搜索，使算法可扩展至大型语料库。

C.2. 时间复杂度

高效词表构建算法的时间复杂度可分为**初始化**与**迭代合并**两部分分析。

- **初始化阶段**构建初始最大堆以跟踪共现频率。给定 N 个输入序列（平均长度为 L ），统计每个大小为 m 的集合内全部 $O(m^2)$ 个词元对的共现，需要 $O(NLm^2)$ 时间。

- **迭代合并阶段**动态处理所涉及的序列。所有迭代中此类序列的总数约为

$$O\left(\frac{N}{|\mathcal{V}_0|}\right) + O\left(\frac{N}{|\mathcal{V}_0|+1}\right) + \dots + O\left(\frac{N}{Q}\right) \simeq O(\log QN).$$

对每个序列，更新链表需 $O(Lm)$ 时间，统计共现需 $O(Lm^2)$ 时间，将共现插入最大堆至多需 $O(Lm^2 \log H)$ 时间。其中 H 为堆大小，至多为 $O(NLm)$ 。因此迭代合并的总体时间复杂度为

$$O(\log QN(Lm + Lm^2 + Lm^2 \log H)) = O(\log Q \log H \cdot NLm^2).$$

因此，所提出词表构建算法的总体时间复杂度为 $O(\log Q \log H \cdot NLm^2)$ ，其中迭代合并阶段占主导，显著优于朴素词表构建的 $O(QNLm^2)$ 。

D. 补充相关工作

将 LLM 与用户动作对齐。开发动作词元化方法的一项核心动机，是高效、有效地将预训练生成模型 (e.g., LLM (OpenAI, 2022; Anil et al., 2023; Touvron et al., 2023; Zhao et al., 2023)) 与用户动作数据对齐 (Geng et al., 2022; Zheng et al., 2024; Hou et al., 2025)。典型流程先将动作序列词元化为动作词元，再对融合语言词元与动作词元的输入进行指令微调。词元化策略的选择至关重要，主要有三种范式：

- **每个物品一个词元：**每个动作由单个词元表示，得到的稠密向量通常来自预训练语义编码器或可学习嵌入表 (Hou et al., 2022; Liao et al., 2024; Zhu et al., 2024b; Zhang et al., 2025b; Kim et al., 2024)。该方法的序列长度高效，但面临内存与可扩展性问题，尤其是唯一物品数常超过 LLM 的典型词表大小；将 LLM 与如此大的词表对齐会带来工程和优化挑战。
- **基于文本的词元化：**将每个动作表示为文本字符串，可自然对齐 LLM 的原生模态 (Geng et al., 2022; Zhang et al., 2025a; Bao et al., 2023)，但会显著延长词元序列，降低词元化效率并增加推理延迟。
- **离散词元：**将动作词元化为来自紧凑共享词表的少量离散词元 (Zheng et al., 2024; Tan et al., 2024; Jin et al., 2024)。该方法兼顾序列长度与内存效率，是构建基于 LLM 的推荐系统的实用选择。

E. 补充讨论

E.1. 集合置换正则化的优势

SPR 从多个方面使模型受益：

- **词元利用率视角。**SPR 可避免单个动作的特征总被合并为最压缩的高层词元，而是根据置换与合并规则，将动作词元化为高层和低层词元，从而增加训练与推理中实际参与的词元数。如 Figure 5 与 Section 4.4.2 所示，SPR 使词元利用率在第 5 轮由 56.89% 提升至 95.33%，表明更多词元得到训练。
- **数据增强视角。**SPR 丰富了可用于模型训练的词元序列。没有 SPR 时，每个动作序列只能得到一个固定词元序列；SPR 则允许多种词元化方式 (见 Figure 1)。这些增强序列保持相同语义，却让模型接触更丰富的词元模式，从而改善泛化能力，Table 5 中变体 (3.1) 的性能对此提供了证据。
- **集成视角。**SPR 还支持推理时增强：同一输入动作序列可增强为多个词元序列，每个序列可能产生不同的下一物品排序。集成这些推荐结果可改善总体性能，见 Table 5 中变体 (3.2) 及 Figure 6。

E.2. ActionPiece 与特征交叉的联系

排序模型的核心组成之一，是自动捕获有助于推荐性能的特征交叉模式 (Shan et al., 2016; Wang et al., 2017; 2021; Bian et al., 2022)，这在概念上与 ActionPiece 相关。关键差异在于交互建模层级：以往工作通常在模型层自动进行特征交叉，通过网络架构隐式学习交互；ActionPiece 则在数据层合并特征。

虽然未直接对比这些特征交叉模型，我们纳入了遵循类似设计理念的多个基线。例如，Table 4 中的 HSTU 与 Table 5 中的变体 (2.1) 使用与 ActionPiece 相同的底层物品特征，但以展平、未合并的形式输入。在这些设定中，特征交互由自回归模型通过自注意力和前馈层在模型层学习。

结果表明，在数据层执行特征合并的 ActionPiece，在推荐质量 (Tables 4 and 5) 与效率 (Figure 4) 上始终优于这些方法。这一点在归一化序列长度 (NSL) 上尤为明显：HSTU 与变体 (2.1) 的 NSL 均为 1，说明其序列长于 ActionPiece 词元化得到的紧凑序列。

F. 数据集

类别。“Sports”和“Beauty”是评估生成式推荐模型的两个常用基准 (Rajput et al., 2023; Jin et al., 2024; Hua et al., 2023)，我们在其上实验以确保与现有结果公平比较。此外，引入“CDs”，其交互量约为“Sports”的 4 $\times$ ，可用于在更大数据集上评估 GR 模型的可扩展性。由于目前“CDs”尚无公开的生成式推荐结果，我们严格遵循“Sports”和“Beauty”等公开基准的实验设置以确保公平比较。

序列截断长度。遵循 Rajput et al. (2023)，过滤评论少于 5 条的用户，并将包括 ActionPiece 在内的“生成式”方法的动作序列最大长度截断为 20。对于“基于 ID”及“特征 + ID”基线，按其原论文建议将最大长度设为 50。

物品文本特征。遵循 Rajput et al. (2023); Zheng et al. (2024); Hou et al. (2024a); Sheng et al. (2025)，特征工程首先将每个物品的多个原始文本特征组合成一个句子，再用预训练句子嵌入模型编码为向量表示。所有实现均拼接标题、价格、品牌、特征、类别与描述，并使用 sentence-t5-base (Ni et al., 2022) 作为句子嵌入模型。

- 编码后的 768 维句子嵌入直接作为 UniSRec 的文本物品表示。
- 使用残差量化 (RQ) (Rajput et al., 2023; Zeghidour et al., 2021; Zheng et al., 2025) 将句子嵌入量化为三个码，每个码有 256 个候选；另加一个标识码防止冲突。这四个码共同作为 TIGER 与 SPM-SID 基于 RQ 的语义 ID。
- 对 FDSA、S³-Rec、VQ-Rec、HSTU 及本文方法等需要物品特征的基线，遵循 Hou et al. (2023)，使用优化乘积量化 (OPQ) (Ge et al., 2013) 量化句子嵌入。除按原论文建议将 VQ-Rec 的句子嵌入量化为 32 个码外，其余方法均量化为 4 个码以确保公平比较，每位码的码本大小为 256。对生成式方法 HSTU 与 ActionPiece，还加入额外标识码防止冲突。与基于 RQ 的语义 ID 不同，乘积/向量量化产生的特征不要求特定顺序。

G. 基线

我们将 ActionPiece 与以下代表性基线比较：

G.1. 基于 ID 的序列推荐方法

- SASRec (Kang & McAuley, 2018) 使用唯一物品 ID 表示每个物品，以自注意力 Transformer 解码器编码物品 ID 序列，并通过优化二元交叉熵目标训练。
- BERT4Rec (Sun et al., 2019) 同样使用唯一物品 ID 表示每个物品，但不同于 SASRec，它用双向 Transformer 编码器编码物品 ID 序列，并采用掩码预测目标训练。

G.2. 特征增强的序列推荐方法

- FDSA (Zhang et al., 2019) 将物品特征嵌入与普通注意力层融合以获得特征表示，随后通过自注意力块分别处理物品 ID 序列与特征序列。
- S³-Rec (Zhou et al., 2020) 首先以自监督预训练捕获物品特征与物品 ID 的相关性，再加载检查点，仅使用物品 ID 针对下一物品预测微调。
- VQ-Rec (Hou et al., 2023) 使用预训练语言模型将文本特征编码为稠密向量，再用乘积量化将其转换为语义 ID，并池化语义 ID 嵌入以表示物品。由于实验并非迁移学习设定，我们省略原论文的两阶段训练策略，复用其模型架构，从头训练，并使用批内对比损失，批大小为 256。

G.3. 生成式推荐方法

每个生成式推荐基线都对应 Table 1 中的一种动作词元化方法。

- P5-CID (Hua et al., 2023) 是将推荐任务表述为文本到文本格式的 P5 (Geng et al., 2022) 的扩展。本文将其作为基于层次聚类的代表性动作词元化方法：它将用户—物品交互拉普拉斯矩阵的特征向量组织为层次结构，并以各层聚类 ID 作为物品索引。实现时采用与 ActionPiece 相同的骨干（从头训练的编码器—解码器 Transformer），并使用 P5-CID 生成的索引。
- TIGER (Rajput et al., 2023) 与 VQ-Rec 类似地编码文本特征，但使用 RQ-VAE 将其量化为语义 ID。模型自回归预测下一语义 ID，并使用束搜索推理。束大小设为 50，以生成 top- K 推荐。
- LMIndexer (Jin et al., 2024) 以文本为输入并预测语义 ID。首先用文本词元化器处理每个物品的文本描述，再拼接所得文本词元以构成输入动作序列，并用自监督目标学习目标物品的语义 ID。Table 4 中结果取自原论文；由于其在与其他方法相似的计算预算下无法在大型“CDs”数据集上收敛，故不报告该结果。
- HSTU (Zhai et al., 2024) 将原始物品特征离散化为词元，作为生成式推荐的输入词元，并提出同时改善性能与效率的轻量 Transformer 层。动作词元化使用与本文相同的物品特征，并按特定顺序排列成各物品的词元。
- SPM-SID (Singh et al., 2024) 首先将每个物品词元化为语义 ID，再使用 SentencePiece 模型 (SPM) (Kudo & Richardson, 2018) 将物品内部重要的语义 ID 模式合并为词表中的新词元。原论文将其用于排序模型，本文则适配至生成式推荐：拼接 SPM 词元作为输入，送入 T5 模型，并自回归生成 SPM 词元作为推荐结果。

Table 7. ActionPiece 在各数据集上的超参数设置。

超参数	Sports	Beauty	CDs
learning_rate	0.005	0.001	0.001
warmup_steps	10,000	10,000	10,000
dropout_rate	0.1	0.1	0.1
weight_decay	0.15	0.15	0.07
vocabulary_size	40,000	40,000	40,000
n_inference_segments	5	5	5
beam_size	50	50	50
num_layers	4	4	4
d_model	128	128	256
d_ff	1,024	1,024	2,048
num_heads	6	6	6
d_kv	64	64	64
optimizer	adamw	adamw	adamw
lr_scheduler	cosine	cosine	cosine
train_batch_size	256	256	256
max_epochs	200	200	200
early_stop_patience	20	20	20

H. 实现细节

基线。 BERT4Rec、SASRec、FDSA、S³-Rec、TIGER 与 LMIndexer 在 “Sports” 和 “Beauty” 基准上的结果直接取自现有论文 (Zhou et al., 2020; Rajput et al., 2023; Jin et al., 2024)。其他结果由我们谨慎实现基线并按原论文建议调优超参数获得。BERT4Rec、SASRec、FDSA 与 S³-Rec 使用开源推荐库 RecBole (Zhao et al., 2021) 实现；其他方法使用 HuggingFace Transformers (Wolf et al., 2020) 与 PyTorch (Paszke et al., 2019) 自行实现，并使用 FAISS (Douze et al., 2024) 量化句子表示。

ActionPiece。采用类似 T5 (Raffel et al., 2020) 的编码器—解码器 Transformer 架构，编码器和解码器均为四层。多头注意力包含六个头，每头维度为 64。对 “Sports” 和 “Beauty”，遵循 Rajput et al. (2023)，将词元嵌入维度设为 128、中间前馈层维度设为 1024，共有 4.46M 个非嵌入参数。对更大的 “CDs”，相应维度设为 256 与 2048，共有 13.11M 个非嵌入参数。推理使用束大小为 50 的束搜索。P5-CID、TIGER 与 SPM-SID 使用相同模型架构，仅动作词元化方法不同。ActionPiece 推理时 SPR 分词结果数设为 $q = 5$ ，词表大小在 $\{5k, 10k, 20k, 30k, 40k\}$ 中调优。

训练。 GR 模型从头训练至多 200 轮；若验证集 NDCG@10 连续 20 轮未改善，则提前停止。批大小为 256，学习率从 $\{1 \times 10^{-3}, 3 \times 10^{-3}, 5 \times 10^{-3}\}$ 中选择，预热步数为 10,000。dropout 率为 0.1，权重衰减从 $\{0.07, 0.1, 0.15, 0.2\}$ 中调优。所有自行实现的方法均使用随机种子 $\{2024, 2025, 2026, 2027, 2028\}$ 重复实验五次，选取验证集平均 NDCG@10 最佳的模型检查点在测试集上评估并报告。每个模型均在单张 40G NVIDIA A100 GPU 上训练。

推理。 ActionPiece 的推理流程与 TIGER 相同。解码器自回归生成表示目标物品的词元序列。训练时以原始物品特征作为标签，不进行增强或词元合并。推理时，对每个测试样例的增强版本使用束搜索生成排名靠前的词元序列；将概率最高的序列（即前缀）保留在束中（束大小见 Table 7），模型逐个生成词元，直

至达到目标序列长度。

```

1 def vocab_construction_iteration(max_heap, vocab, rules, pair2head):
2     """Performs one iteration of efficient vocabulary construction.
3
4     Args:
5         max_heap (PriorityQueue): Max-heap storing (co_occurrence, (c_u, c_v)) pairs.
6         vocab (List[Tuple]): Current vocabulary with merge rules.
7         rules (Dict): Merge rule {(c_u, c_v): c_new} mapping.
8         pair2head (Dict): Inverted indices that store mappings from the token pair to
9             all sequences that contain this token pair.
10
11     """
12     # Get most frequent valid pair (c_u, c_v) from max-heap
13     # Efficient version of "Count" in Algorithm 1
14     # Avoid recalculating co-occurrences for each iteration, by maintaining them in a
15     # max-heap with the lazy update trick
16     while not max_heap.empty():
17         co_occurrence, (c_u, c_v) = max_heap.get()
18         if not is_outdated((c_u, c_v), co_occurrence): # Outdated values are lazily
19             removed.
20             break
21
22     # Create new token and update vocabulary
23     c_new = len(vocab)
24     vocab.append((c_u, c_v))
25     rules[(c_u, c_v)] = c_new
26
27     # Update sequences containing (c_u, c_v)
28     seq_ids = pair2head[(c_u, c_v)].copy() # IDs of affected sequences
29     delta_counts = defaultdict(int)
30
31     for sid in seq_ids:
32         # Merge all (c_u, c_v) pairs in sequence
33         new_seq = merge_sequence(seqs[sid], c_u, c_v, c_new) # Replace pairs
34         seqs[sid] = new_seq
35
36         # Calculate pair co-occurrence changes
37         new_freqs = count_pairs(new_seq) # Get token co-occurrences of the updated
38             sequence
39         delta = diff_counts(new_freqs, old_freqs[sid]) # Compute co-occurrence
40             differences
41         update_index(pair2head, delta, sid) # Update inverted index
42         old_counts[sid] = new_freqs
43
44         # Accumulate global co-occurrence changes
45         for p, cnt in delta.items():
46             delta_counts[p] += cnt
47
48     # Lazy update max-heap with new co-occurrences
49     for (c_u, c_v), delta in delta_counts.items():
50         if abs(delta) < eps: continue # eps: minimum update threshold
51         all_pair_freqs[(c_u, c_v)] += delta # Global co-occurrences
52         max_heap.put( (-all_pair_freqs[(c_u, c_v)], (c_u, c_v)) )

```

Figure 7. 高效词表构建算法单轮迭代的伪代码，展示如何使用带惰性更新的最大堆跟踪并合并高频词元对。